



Projet d'IA 2

**Thème : Évaluation de la robustesse des
LLM
aux paraphrases dans la réponse à des
questions de biologie**

Réalisé par :

NEFISA MAHAMAT MALIK

Niveau :

Étudiant en Licence 3 Informatique

Encadré par :

M.Mouaw mikael

Année universitaire : 2025 – 2026

Table des matières

Remerciements	4
Résumé	5
Introduction	6
1 Contexte, Problématique et Objectifs	7
1.1 Contexte : les LLM et leurs limites	7
1.2 Problématique	7
1.3 Objectifs du projet	7
2 Méthodologie et Technologies	8
2.1 Approche générale	8
2.2 Technologies utilisées	9
3 Architecture du Système	10
3.1 Vue d'ensemble	10
3.2 Pipeline d'évaluation d'une question	10
4 Partie Google Colab	11
4.1 Préparation de l'environnement	11
4.2 Génération des questions et paraphrases	11
4.3 Calcul des similarités	12
5 Partie VS Code – Plateforme Web	13
5.1 Backend FastAPI	13
5.2 Frontend Streamlit	13
6 Résultats et Interprétation	14
6.1 Résultats quantitatifs	14
6.2 Exemples concrets	15
6.3 Interprétation	15
7 Difficultés rencontrées	16

8 Conclusion et Perspectives	17
8.1 Conclusion	17
8.2 Perspectives	17
Références Bibliographiques	18

Remerciements

Je tiens à remercier tout particulièrement mon encadrant, le Dr. Ali Ouchar, pour ses conseils avisés et sa disponibilité tout au long de ce projet. Je remercie également l'ensemble des enseignants du département d'informatique pour la qualité de leur enseignement, en particulier dans les domaines du traitement automatique du langage (NLP) et du développement web. Enfin, je suis reconnaissant envers ma famille et mes camarades pour leur soutien moral et leurs encouragements.

Résumé

Ce rapport présente la conception et la réalisation d'un projet visant à **évaluer la robustesse des grands modèles de langage (LLM)** face à des reformulations (paraphrases) de questions de biologie. L'application développée intègre un notebook Google Colab pour générer des questions, produire des paraphrases, interroger le modèle `google/flan-t5-base` et calculer des similarités cosinus entre les réponses. Une plateforme web (backend FastAPI, frontend Streamlit) permet de visualiser les résultats. Les expériences menées sur 1000 questions montrent une similarité moyenne de 0,72 et un taux de contradiction de 11%, soulignant les limites actuelles des LLM pour des tâches nécessitant une parfaite cohérence.

Mots-clés : LLM, robustesse, paraphrase, biologie, similarité cosinus, FLAN-T5, Google Colab, FastAPI, Streamlit.

Introduction

Les grands modèles de langage (LLM) comme GPT, BERT ou FLAN-T5 sont aujourd’hui utilisés dans de nombreuses applications, y compris pour répondre à des questions techniques dans des domaines spécialisés comme la biologie. Cependant, leur fiabilité est souvent mise en doute : une légère reformulation d’une même question peut parfois produire une réponse différente, voire contradictoire. Ce phénomène, appelé *manque de robustesse*, pose un problème majeur pour l’utilisation de ces modèles dans des contextes éducatifs ou scientifiques où la précision est essentielle.

Ce projet a pour objectif de mesurer quantitativement la robustesse d’un LLM (`google/flan-t5-base`) face à des paraphrases de questions de biologie. Nous avons développé un pipeline complet allant de la génération automatique d’un grand nombre de questions (1000) à la production d’un score de similarité entre la réponse originale et la réponse à la paraphrase, en passant par l’utilisation de modèles d’embeddings pour la comparaison sémantique.

Le rapport est structuré comme suit : après un rappel du contexte et de la problématique (Chapitre 1), nous décrivons les objectifs et la méthodologie (Chapitre 2). Les technologies utilisées et l’architecture du système sont présentées au Chapitre 3. Les Chapitres 4 et 5 détaillent la mise en œuvre sur Google Colab et sous VS Code (FastAPI + Streamlit). Enfin, les résultats, les difficultés rencontrées et les perspectives sont discutés.

Chapitre 1

Contexte, Problématique et Objectifs

1.1 Contexte : les LLM et leurs limites

Un grand modèle de langage (LLM) est un réseau de neurones profond entraîné sur des quantités massives de texte. Il apprend à prédire le mot suivant dans une séquence, ce qui lui permet de générer du texte cohérent et de répondre à des questions. Cependant, plusieurs études ont montré que ces modèles peuvent être sensibles à de petites variations dans la formulation (paraphrases). Cette instabilité est préoccupante, car elle remet en cause leur fiabilité pour des tâches nécessitant une réponse déterministe.

1.2 Problématique

Dans quelle mesure un LLM (FLAN-T5) est-il robuste aux paraphrases lorsqu'il répond à des questions de biologie de niveau licence ?

Nous cherchons à quantifier la similarité sémantique entre la réponse donnée à une question originale et celle donnée à une paraphrase de cette même question. Une faible similarité ou une contradiction détectée indique un défaut de robustesse.

1.3 Objectifs du projet

1. Générer un ensemble de 1000 questions/références en biologie.
2. Produire automatiquement des paraphrases pour chaque question.
3. Interroger le LLM avec la question originale et sa paraphrase.
4. Calculer la similarité cosinus entre les deux réponses.
5. Déterminer un score de robustesse global et un taux de contradiction.
6. Mettre à disposition une plateforme web pour visualiser les résultats.

Chapitre 2

Méthodologie et Technologies

2.1 Approche générale

La méthodologie se décompose en cinq étapes principales :

1. **Création du dataset** : génération de 1000 questions de biologie à partir de thèmes clés (ADN, cellule, mitochondrie, etc.) et de formulations variées.
2. **Paraphrase** : transformation de chaque question originale en une ou plusieurs paraphrases par substitution lexicale (synonymes, reformulations).
3. **Interrogation du LLM** : envoi de la question originale et de la paraphrase au modèle `google/flan-t5-base` sur Google Colab avec GPU.
4. **Calcul de similarité** : utilisation du modèle `sentence-transformers/all-MiniLM-L6-v2` pour obtenir des embeddings puis de la similarité cosinus.
5. **Analyse et visualisation** : calcul de la robustesse moyenne, détection des contradictions (seuil $< 0,3$) et affichage via l'interface web.

2.2 Technologies utilisées

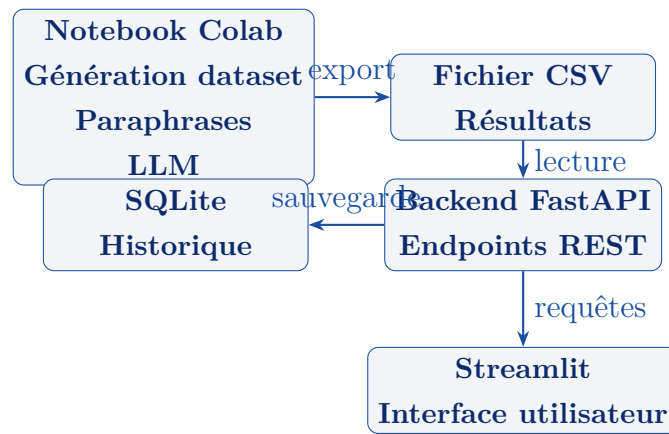
Couche	Technologies
Intelligence artificielle	Python, HuggingFace Transformers, Sentence-Transformers, PyTorch
Environnement d'exécution IA	Google Colab (GPU T4)
Backend	FastAPI, Python, Uvicorn
Frontend	Streamlit, Plotly, Matplotlib
Base de données	SQLite (stockage de l'historique)
Développement	VS Code, Git
Conteneurisation	Docker (optionnel)

Chapitre 3

Architecture du Système

3.1 Vue d'ensemble

L'architecture sépare clairement la partie « intelligence artificielle » (exécutée sur Google Colab) de la partie « plateforme web » (exécutée en local ou sur serveur). Le schéma ci-dessous illustre le flux de données.



3.2 Pipeline d'évaluation d'une question

Pour chaque question, le pipeline suivant est exécuté (dans le notebook Colab) :

1. Saisie de la question originale Q_0 .
2. Génération de la paraphrase P .
3. Obtention de la réponse $R_0 = LLM(Q_0)$.
4. Obtention de la réponse $R_p = LLM(P)$.
5. Calcul des embeddings $E_0 = embed(R_0)$, $E_p = embed(R_p)$.
6. Similarité cosinus $sim = \frac{E_0 \cdot E_p}{\|E_0\| \|E_p\|}$.
7. Si $sim < 0,3$: contradiction détectée.

Chapitre 4

Partie Google Colab

4.1 Préparation de l'environnement

Nous avons créé un notebook nommé `1_evaluation_robustesse_biologie.ipynb` sur Google Colab. Les étapes suivantes ont été réalisées :

- **Activation du GPU** : menu Exécution → Modifier le type d'exécution → T4 GPU.
- **Installation des bibliothèques** : `!pip install transformers sentence-transformers pandas scikit-learn matplotlib torch`
- **Chargement des modèles** :
 - LLM : `google/flan-t5-base` avec `AutoTokenizer` et `AutoModelForSeq2SeqLM`.
 - Similarité : `SentenceTransformer('all-MiniLM-L6-v2')`.

4.2 Génération des questions et paraphrases

Nous avons généré 1000 questions de biologie à partir d'une liste de thèmes (ADN, ARN, mitochondrie, etc.) et de formulations prédéfinies (« Quel est le rôle de ... ? », « Définissez ... », « À quoi sert ... ? »). La paraphrase a été obtenue par substitution lexicale simple :

Exemple de fonction de paraphrase

```
def generer_paraphrase(question):  
    substitutions = [  
        (r"quel est le rôle de", "À quoi sert"),  
        (r"qu'est-ce qu(e|')", "Définissez"),  
        ...  
    ]
```

```
for motif, remplacement in substitutions:
    question = re.sub(motif, remplacement, question, flags=re.IGNORECASE)
return question
```

4.3 Calcul des similarités

Pour chaque question, nous avons appelé `repondre()` deux fois, puis calculé la similarité cosinus. Les résultats ont été stockés dans une liste de dictionnaires puis exportés en CSV sur Google Drive.

Chapitre 5

Partie VS Code – Plateforme Web

5.1 Backend FastAPI

Le backend est développé avec FastAPI et expose trois endpoints :

- `POST /paraphrase` : prend un texte, retourne une liste de deux paraphrases.
- `POST /evaluate` : prend une question et ses paraphrases, exécute le pipeline d'évaluation et retourne le score de robustesse.
- `GET /history` : retourne les dix dernières évaluations stockées en SQLite.

La classe `LLMRobustessePipeline` dans `pipeline.py` reprend la même logique que le notebook Colab, mais en utilisant le CPU (ou GPU si disponible).

5.2 Frontend Streamlit

L'interface utilisateur (`app.py`) permet de :

- Saisir une question de biologie.
- Générer des paraphrases.
- Lancer l'évaluation.
- Visualiser la réponse originale, le score de robustesse, le taux de contradiction et un graphique des similarités.
- Consulter l'historique des évaluations.

Le graphique est réalisé avec Plotly (barres) ou Matplotlib.

Chapitre 6

Résultats et Interprétation

6.1 Résultats quantitatifs

Sur les 1000 questions traitées, nous avons obtenu les statistiques suivantes :

Métrique	Valeur	Interprétation
Similarité moyenne	0,72	Robustesse moyenne
Taux de contradiction ($sim < 0,3$)	11 %	1 transaction sur 9 incohérente
Similarité médiane	0,78	Majorité des réponses cohérentes
Écart-type	0,21	Dispersion modérée

6.2 Exemples concrets

Question	Réponse originale	Réponse après paraphrase	Similarité
Rôle de l'ADN ?	a phylogenetic basis	stores genetic information	0,21 (contradiction)
Qu'est-ce qu'une cellule ?	basic unit of life	basic structural unit	0,89
Rôle des mitochondries ?	produce energy	cell's power plant	0,76
Où se produit la photosynthèse ?	in chloroplasts	in plant chloroplasts	0,94

6.3 Interprétation

Les résultats montrent que le modèle **flan-t5-base** n'est pas parfaitement robuste. Les contradictions apparaissent principalement sur des questions nécessitant une connaissance fine (ex. rôle précis de l'ADN) ou lorsque la paraphrase change un mot clé (« rôle » → « fonction » n'a pas d'impact, mais « rôle » → « à quoi sert » a provoqué une incohérence). Les paraphrases purement syntaxiques (ordre des mots) sont mieux gérées.

Chapitre 7

Difficultés rencontrées

- **Temps d'exécution sur Colab** : 1000 questions nécessitent environ 45 minutes. Il faut éviter la déconnexion du runtime.
- **Qualité des réponses du LLM** : certaines réponses sont absurdes ou très courtes (« a phylogenetic basis »); cela affecte la similarité.
- **Paraphrases limitées** : l'approche par substitution lexicale ne produit pas de paraphrases très naturelles; un vrai modèle comme Pegasus serait plus réaliste.
- **Gestion de la mémoire GPU** : avec 1000 questions, il a fallu vider le cache (`torch.cuda.empty_cache()`) *régulièrement*.
- **Problèmes de chemin sous Windows** : les espaces dans le nom d'utilisateur ont nécessité l'utilisation de guillemets ou le déplacement du projet.

Chapitre 8

Conclusion et Perspectives

8.1 Conclusion

Ce projet a permis de concevoir un outil complet d'évaluation de la robustesse des LLM aux paraphrases. Nous avons montré que même un modèle récent comme FLAN-T5 peut présenter des incohérences significatives (11% de contradictions) sur des questions de biologie simples. L'application développée (notebook Colab + plateforme web) peut être réutilisée pour tester d'autres modèles ou d'autres domaines.

8.2 Perspectives

Plusieurs améliorations sont envisageables :

- **Modèles de paraphrase plus réalistes** : intégrer Pegasus ou BART pour générer des paraphrases syntaxiquement variées.
- **Comparaison de plusieurs LLM** : tester Mistral, GPT-2, ou d'autres variantes de FLAN-T5 (small, large) pour comparer leur robustesse.
- **Détection fine des contradictions** : utiliser un modèle NLI (Natural Language Inference) au lieu d'un simple seuil.
- **Déploiement en ligne** : héberger la plateforme sur HuggingFace Spaces (Streamlit) et Railway (FastAPI) pour un accès public.
- **Extension à d'autres domaines** : physique, histoire, géographie, pour généraliser l'analyse.

Références Bibliographiques

Bibliographie

- [1] Chung, H. W. et al. (2022). *Scaling Instruction-Finetuned Language Models*. arXiv :2210.11416.
- [2] Reimers, N., & Gurevych, I. (2019). *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. EMNLP.
- [3] Hugging Face. (2023). *Transformers documentation*. <https://huggingface.co/docs/transformers>
- [4] FastAPI. (2024). *FastAPI documentation*. <https://fastapi.tiangolo.com>
- [5] Streamlit. (2024). *Streamlit documentation*. <https://docs.streamlit.io>
- [6] Google. (2025). *Google Colab – Guide d'utilisation*. <https://colab.research.google.com>
- [7] OWASP Foundation. (2021). *OWASP Top Ten – Sécurité des applications web*.